

PLATEFORME D'APPRENTISSAGE · v1.0

MNEMO

Réviser par missions narratives.

Capturer ses compétences en agents-mémoire.

Mnemo transforme chaque cours que vous apprenez en une mission de codage interactive, puis capture votre compétence dans un agent-mémoire interrogeable. Votre réseau grandit au fur et à mesure — chaque agent est un snapshot de votre savoir à un moment précis, dans un domaine précis.

PYTHON

SQL

IA

DATA SCIENCE

DEV WEB

DOCUMENT

Architecture v1.0

AUDIENCE

Apprenant solo · Concepteur

STATUT

MVP fonctionnel

1. Concept central

1. Concept central

Mnemo transforme la révision technique (Python, SQL, IA, data science, dev web) en un cycle **apprentissage → mission interactive → agent-mémoire persistant**. L'objectif n'est pas de "savoir plus", mais de **ne jamais perdre ce que vous avez appris** — en ayant un répertoire d'agents qui incarnent chaque étape de votre progression, et que vous pouvez réveiller à tout moment pour réviser sans tout reparcourir depuis zéro.

Deux moteurs complémentaires

Le système repose sur deux moteurs qui travaillent ensemble pour transformer un contenu de cours brut en un artifact de mémoire permanent. Le premier moteur est le générateur de missions, qui convertit un contenu pédagogique (lien DataCamp, blog, vidéo, ou notes libres) en exercice de code scénarisé, strictement borné au périmètre du chapitre soumis. Le second moteur est l'agent-mémoire, qui capture un "snapshot" de compétence sous forme d'agent IA interrogeable, qui répond comme vous, à ce niveau-là, à ce moment-là de votre apprentissage.

MOTEUR 1	MOTEUR 2
Générateur de Missions	Agent-Mémoire
Entrée : lien de cours ou résumé texte Sortie : mission scénarisée = énoncé narratif + exercice de code + critères de validation Rôle : convertir un chapitre en défi interactif borné	Déclencheur : fin d'un cursus complet (niveau 3 atteint) Sortie : snapshot de compétence interrogeable Rôle : capturer et préserver votre savoir dans un domaine précis

La boucle complète se déroule en trois temps. D'abord, vous soumettez le contenu d'un chapitre que vous venez d'étudier (vos notes, un lien DataCamp, une page de documentation). Ensuite, l'IA génère une mission de codage scénarisée qui teste exactement les notions du chapitre, sans jamais déborder sur des concepts non encore vus. Vous codez votre solution dans un éditeur interactif, vous soumettez, et l'IA évalue si votre code répond aux objectifs. Chaque mission validée rapporte de l'XP dans le cursus correspondant. Quand vous atteignez le niveau 3 dans un cursus (1500 XP), un agent-mémoire naît automatiquement dans votre monde virtuel — il devient votre mémoire interrogeable dans ce domaine précis.

2. Pourquoi ça marche pédagogiquement

2. Pourquoi ça marche pédagogiquement

Mnemo s'appuie sur quatre principes pédagogiques éprouvés, combinés dans un cycle cohérent qui maximise la rétention à long terme. Ces principes ne sont pas une innovation théorique mais une synthèse pragmatique de ce qui fonctionne déjà en apprentissage espacé, en formation active et en

gamification sérieuse.

Granularité stricte

Une mission ne couvre jamais plus que le chapitre soumis. Cela évite la surcharge cognitive et la frustration des exercices "qui mélangent tout". Quand l'utilisateur soumet un cours sur les boucles `for` en Python, la mission générée ne fait appel qu'à la syntaxe des boucles `for`, jamais à des fonctions, des classes ou des list comprehensions qui n'ont pas encore été vues. Ce scoping strict est la pièce la plus délicate techniquement : il faut un prompt qui force le modèle à lister explicitement "concepts autorisés" versus "concepts interdits" avant de générer la mission, sinon il y a fuite — le modèle a tendance à enrichir naturellement le périmètre.

Rappel actif espacé (spaced retrieval)

Interroger un agent-mémoire force un rappel actif, bien plus efficace pour la rétention qu'une relecture passive. Quand vous demandez à votre agent "qu'est-ce qu'on a appris la dernière fois ?", il doit reconstruire la réponse à partir de ce qui a été capturé, ce qui force votre cerveau à re-parcourir le chemin de la notion. C'est l'essence du spaced retrieval : tester la mémoire à intervalles réguliers, plutôt que relire passivement. La particularité de Mnemo est que l'agent répond à la première personne, comme votre "vous du futur" — ce qui crée un dialogue intime avec votre propre savoir plutôt qu'avec un tuteur externe.

Scénarisation ludique

Habiller l'exercice en mission narrative (façon GTA) augmente l'engagement et réduit la sensation de "corvée de révision". Au lieu d'un énoncé scolaire sec ("Écrivez une fonction qui calcule la moyenne d'une liste"), l'utilisateur reçoit un briefing narratif ("Un client vient d'appeler. Il lui faut un script qui analyse les ventes du dernier trimestre. Tu as 5 minutes. Prouve que tu mérites ton pay."). La mission est la même, mais l'engagement émotionnel est radicalement différent. La scénarisation transforme la révision en jeu, ce qui augmente la durée des sessions et la fréquence de retour.

Auto-cohérence narrative

Le fait que l'agent "ingénieur Python junior" puisse plus tard postuler auprès de l'agent "ingénieur IA" crée une métaphore visuelle de la progression de carrière — utile pour visualiser où vous en êtes dans un cursus long (ex: devenir ingénieur IA). Cette auto-cohérence narrative transforme l'apprentissage en récit : vous n'êtes plus en train de "faire des exercices", vous êtes en train de faire grandir un réseau de compétences incarnées qui dialoguent entre elles. C'est une différence fondamentale avec les plateformes classiques de flashcards ou de quizzes.

3. Personnas

3. Personnas (si ça devient un produit)

Mnemo a été conçu initialement pour un cas d'usage personnel : un apprenant solo intensif qui étudie plusieurs domaines en parallèle (Python, SQL, IA, dev web) et qui veut garder une trace structurée de sa progression sans tout réapprendre à chaque reprise. Mais l'architecture se prête à plusieurs personnas si le projet évolue vers un produit. Voici les quatre profils types qui pourraient bénéficier de Mnemo, du plus proche du cas initial au plus éloigné.

Persona	Profil	Besoin principal
L'apprenant solo intensif	Apprend plusieurs domaines en parallèle (Python, SQL, IA, dev web)	Ne pas oublier, garder une trace structurée de progression
L'étudiant en reconversion	Suit un bootcamp ou cursus long (6-12 mois)	Motivation, gamification, preuve tangible de compétence acquise
Le formateur / bootcamp	Veut un outil d'évaluation continue pour ses élèves	Suivi de progression par chapitre, missions générées automatiquement
Le recruteur tech (usage dérivé)	Veut évaluer un candidat sur compétences réelles	Voir l'historique d'agents-mémoire = portfolio de compétences vérifiées

4. Architecture du système

4. Architecture du système

L'architecture de Mnemo s'organise en couches successives, depuis l'interface utilisateur jusqu'à la couche narrative de gamification. Chaque couche a une responsabilité claire et peut évoluer indépendamment. Le diagramme ci-dessous présente la vue d'ensemble du système, suivie d'une description détaillée de chaque module.

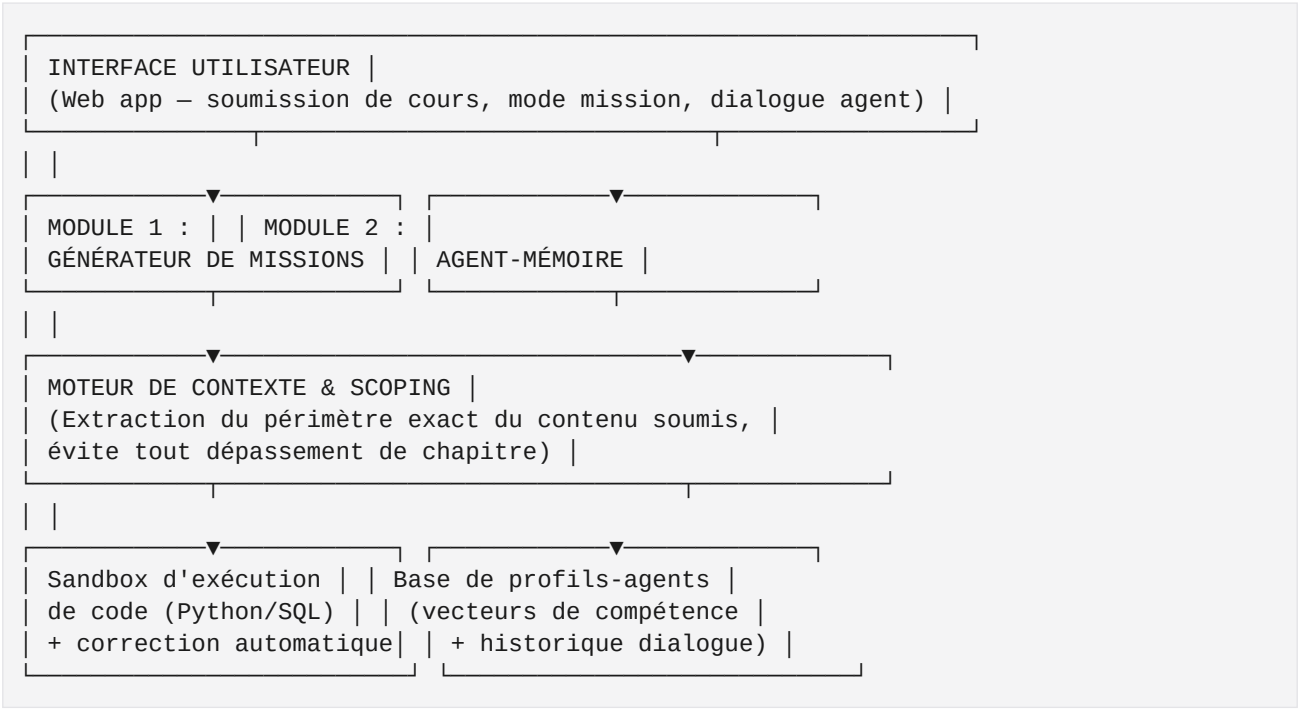


Figure 1 — Vue d'ensemble de l'architecture Mnemo

4.1 Module 1 — Générateur de Missions

Le générateur de missions prend en entrée un lien de cours OU un résumé texte rédigé par l'utilisateur, et produit en sortie une mission scénarisée composée d'un énoncé narratif, d'un exercice de code à compléter, et de critères de validation automatiques. Le pipeline se déroule en cinq étapes successives qui garantissent la qualité et la spécificité de la mission générée.

Étape	Action	Détail
1. Ingestion	Lien → scraping, ou texte libre → utilisation directe	Si lien fourni (DataCamp, doc, blog), extraction du contenu via <code>page_reader</code> . Sinon, utilisation directe du texte soumis.
2. Scoping	Extraction des concepts clés UNIQUEMENT présents dans le contenu	Si le contenu ne parle que de variables et boucles <code>for</code> , la mission ne doit JAMAIS utiliser de fonctions ou de classes.
3. Génération narrative	Prompt structuré → scénario GTA	Ex : "Tu es un agent infiltré qui doit décoder un signal en utilisant uniquement des boucles <code>for</code> et des listes".
4. Génération exercice	Code à compléter + tests de validation	Code de départ minimal (structure), pas de solution complète. Tests automatiques pour valider la soumission.
5. Correction	Exécution en sandbox + feedback contextualisé	Pas juste pass/fail : feedback précis sur ce qui manque, ce qui est bien fait, conseil pour aller plus loin.

Point critique technique : le scoping strict est la pièce la plus délicate. Il faut un prompt qui force le modèle à lister explicitement "concepts autorisés" versus "concepts interdits" avant de générer la mission, sinon il y a fuite. Le modèle a tendance à enrichir naturellement le périmètre (par exemple, si on lui demande une mission sur les variables, il va spontanément ajouter des conditions `if/else` qui n'étaient pas dans le cours). Une validation automatique post-génération peut détecter ces fuites et relancer la génération si nécessaire.

4.2 Module 2 — Agent-Mémoire

L'agent-mémoire se déclenche à la fin d'un cursus complet — pas juste un chapitre. Un agent = un cursus entier (ex: "Introduction à Python", "SQL avancé", "Ingénieur IA"). L'agent contient le résumé structuré de toutes les missions réussies dans ce cursus, le niveau de maîtrise par concept (basé sur le taux de réussite, le nombre de tentatives, la vitesse de résolution), un system prompt généré qui capture "comment vous raisonnez" à ce niveau (style de réponse, erreurs typiques que vous faisiez, façon de nommer les variables, etc.), et un horodatage — c'est un snapshot, pas un agent qui continue d'apprendre seul.

L'interaction se fait par dialogue libre : vous posez une question à l'agent, il répond avec les connaissances et le style que vous aviez à ce moment précis, pas avec les connaissances actuelles du modèle sous-jacent. Cela permet de vérifier "est-ce que j'ai vraiment intégré ça, ou est-ce que j'ai juste copié un exercice". L'agent est votre miroir pédagogique — il vous renvoie votre propre niveau, ce qui est à la fois réconfortant (il ne vous juge pas) et exigeant (il sait exactement où vous en êtes).

4.3 Couche narrative / multi-agents (V2+)

Cette couche n'est pas dans le MVP mais constitue l'extension naturelle du concept. Les agents-mémoire de cursus différents peuvent interagir entre eux dans un scénario : par exemple, l'agent "Python débutant" simule un entretien avec l'agent "recruteur IA". L'usage est l'auto-évaluation par simulation, et la transition entre cursus rendue visible et ludique. Techniquement, cela correspond à un système multi-agent où chaque agent a un system prompt distinct + sa propre mémoire de cursus, orchestré par un agent "modérateur" qui gère le tour de parole. La vue "Monde des Agents" déjà implémentée dans le MVP visualise ces communications potentielles sous forme de réseau.

5. Modèles de données

5. Modèles de données

Le schéma de données de Mnemo est volontairement simple, organisé autour de cinq entités principales qui reflètent fidèlement les concepts pédagogiques. La base actuelle utilise SQLite via Prisma ORM, mais le schéma est portable vers PostgreSQL sans modification structurelle si le projet évolue vers un usage multi-utilisateur. Voici la définition des entités telles qu'implémentées dans le MVP.

Curriculum

L'unité de plus haut niveau. Un cursus regroupe plusieurs missions et peut donner naissance à un agent-mémoire quand il est poussé jusqu'au niveau 3. Les champs principaux sont : `id`, `name` (ex: "Python"), `domain` (enum : python, sql, ai-engineering, data-science, web-dev), `description`, `icon`, `color`, `xp` (cumul XP), `level` (calculé depuis XP, 1 niveau = 500 XP), et une relation vers ses missions et son agent éventuel.

Mission

Un exercice généré et scénarisé. Chaque mission est rattachée à un cursus et contient : `title` (court, percutant), `briefing` (récit narratif style GTA, 2-4 phrases), `objectives` (JSON array des objectifs à valider), `starterCode` (code de départ minimal), `hint` (indice optionnel), `difficulty` (rookie | pro | elite, détermine l'XP : 100/200/300), `language` (Python, SQL, JavaScript), `sourceContent` (le cours original qui a généré la mission), et `status` (active | completed).

MissionSubmission

Une tentative de l'utilisateur pour résoudre une mission. Chaque soumission est évaluée par l'IA et stockée pour historique. Contient : `code` (le code soumis), `feedback` (feedback textuel de l'évaluateur IA), `passed` (booléen), `createdAt`. Une mission est marquée "completed" dès qu'une soumission passe avec succès.

Agent

Le snapshot de compétence interrogeable. Un agent naît automatiquement quand un cursus atteint le niveau 3. Contient : `name` (nom de code stylé, ex: "PY-7", "SQL-Oracle"), `persona` (system prompt qui capture le style et le niveau), `skills` (JSON array des compétences maîtrisées), `totalXp`, `level`, `status` (idle | working | interviewing | relaxing), `activity` (description humaine de l'activité actuelle). L'agent a une relation 1:1 avec son cursus.

AgentConversation

L'historique des messages échangés avec un agent. Contient : agentId, rôle (user | agent), content, createdAt. Cet historique est utilisé pour alimenter le contexte du LLM à chaque nouveau message, ce qui permet à l'agent de "se souvenir" de la conversation en cours et de maintenir une cohérence narrative avec l'utilisateur.

6. Choix techniques

6. Choix techniques recommandés

Le MVP actuel de Mnemo est implémenté en Next.js 16 + TypeScript + Tailwind CSS + Prisma (SQLite) + z-ai-web-dev-sdk. Ce choix a été fait pour pouvoir itérer rapidement sur l'interface et profiter de l'écosystème React. Si le projet évolue vers un produit SaaS, une migration progressive vers une architecture Python (FastAPI) est recommandée pour cohérence avec l'apprentissage Python de l'utilisateur et pour profiter de l'écosystème data science mature. Voici les choix techniques recommandés par composant pour la version produit.

Composant	Choix recommandé	Justification
Backend	Python (FastAPI)	Cohérent avec l'apprentissage actuel, écosystème mature
Base de données	PostgreSQL	Relationnel adapté aux modèles, bon support JSON pour les champs flexibles
Sandbox d'exécution	Docker isolé / Judge0 (self-hosted)	Exécution sécurisée du code utilisateur, supporte Python et SQL
Génération IA	z-ai-web-dev-sdk (MVP) / Claude (V2)	Déjà intégré, bon raisonnement structuré pour le scoping strict
Frontend	Next.js 16 (MVP) / React (V2)	Standard, bon support pour interface interactive type "mission"
Vectorisation compétences	Embeddings simples + scoring manuel au début	Pas besoin de ML complexe au MVP, un simple scoring par concept suffit

7. Roadmap par phases

7. Roadmap par phases

La roadmap de Mnemo est conçue pour valider le concept progressivement, sans investir d'infrastructure lourde avant d'avoir confirmé que la boucle pédagogique fonctionne. Chaque phase produit une version utilisable, et la décision de passer à la suivante se prend sur la base de l'usage réel, pas sur un calendrier arbitraire.

Phase 0	Validation du concept (manuel, 1-2 semaines)
	Test "à la main" avec l'IA en conversation : on soumet un résumé de cours, on demande une mission scénarisée, on évalue si le scoping reste propre. Objectif : valider que le scoping strict fonctionne avant de coder quoi que ce soit.
Phase 1	MVP Générateur de Missions (3-4 semaines)
	Interface web simple, soumission de texte de cours (pas encore de scraping de lien), génération de mission + éditeur de code Python uniquement. Pas encore d'agent-mémoire. C'est la version actuellement déployée.
Phase 2	Agent-Mémoire (2-3 semaines)
	À la fin d'un cursus (niveau 3 atteint), génération automatique du system prompt de l'agent. Interface de dialogue avec l'agent-mémoire. Stockage persistant. Cette phase est également déjà implémentée dans le MVP actuel.
Phase 3	Extension multi-domaines (2-3 semaines)
	Support SQL (sandbox SQL + types de missions adaptés), support data science (missions avec datasets, notebooks), scraping de liens de cours (pas seulement texte collé). L'intégration DataCamp via page_reader est déjà fonctionnelle.
Phase 4	Couche narrative / multi-agents (optionnel)
	Scénarios d'interaction entre agents-mémoire. Habillage visuel "monde virtuel" (déjà implémenté en version statique, à étendre avec interactions réelles entre agents).
Phase 5	Validation produit (si pivot SaaS)
	Tester avec 5-10 apprenants externes (amis, communauté dev). Mesurer : rétention de connaissance réelle (test avant/après), engagement, complétion des cursus. Identifier le persona qui paie le plus naturellement.
Phase 6	SaaS (si validation positive)
	Multi-tenant, gestion de comptes, facturation. Dashboard formateur si pivot B2B2C vers les bootcamps. Optimisation des coûts d'API (cache de missions similaires, modèles plus légers pour le scoping).

8. Risques et points d'attention

8. Risques et points d'attention

Tout projet d'apprentissage basé sur l'IA comporte des risques spécifiques liés à la qualité de la génération, au coût des appels API, à la fidélité des agents-mémoire, et au risque de scope-creep si le projet est mené en parallèle d'autres initiatives. Voici les quatre risques principaux identifiés et leurs

stratégies de mitigation.

Risque	Impact	Mitigation
Fuite de scoping (mission dépasse le chapitre)	Frustration, perte de confiance dans l'outil	Prompt strict avec liste explicite concepts autorisés/interdits, validation automatique post-génération
Coût API élevé (génération + correction + dialogue)	Modèle économique fragile en SaaS	Cache de missions pour contenus similaires, modèles moins chers pour le scoping, modèle premium pour la génération créative uniquement
Agent-mémoire devient "obsolète" si pas assez fidèle	L'intérêt principal du produit s'effondre	Bien calibrer le system prompt généré, tester qu'il répond vraiment "à ton niveau" et pas avec les connaissances actuelles du modèle
Scope-creep (projet mené en parallèle d'autres)	Aucun projet n'avance	Garder Mnemo en mode "perso/phase 0-1" tant que les autres projets n'ont pas atteint leur MVP

9. Prochaine étape concrète

9. Prochaine étape concrète recommandée

Le MVP étant désormais fonctionnel (génération de missions, éditeur de code, agents-mémoire, vue Monde, intégration DataCamp), la prochaine étape recommandée est l'utilisation réelle en conditions d'apprentissage. Trois axes prioritaires peuvent être explorés en parallèle pour faire passer Mnemo de "prototype fonctionnel" à "outil d'apprentissage quotidien".

Axe 1 — Tester le scoping en conditions réelles

Prendre les 5 prochains chapitres que vous allez étudier (que ce soit sur DataCamp, dans un livre, ou en vidéo) et les soumettre à Mnemo. Pour chacun, vérifier : la mission générée couvre-t-elle exactement le chapitre ? Y a-t-il des fuites (concepts non vus qui apparaissent) ? Le niveau de difficulté est-il adapté ? Ces 5 tests permettront de calibrer le prompt du générateur et d'identifier les cas limites. Gardez une trace écrite des problèmes rencontrés pour itérer sur le prompt.

Axe 2 — Évaluer la fidélité des agents-mémoire

Une fois un premier cursus poussé au niveau 3 et l'agent né, mener une session de questions type : demander à l'agent des rappels sur les notions apprises, lui soumettre un défi, lui demander un conseil. Évaluer : les réponses sont-elles cohérentes avec votre niveau réel ? L'agent donne-t-il l'impression de parler comme vous ? Identifie-t-il correctement vos acquis et vos lacunes ? Cette évaluation qualitative est cruciale : si l'agent ne "sonne" pas juste, la valeur perçue du produit s'effondre.

Axe 3 — Étendre la couverture des langages

Le MVP supporte Python, SQL et JavaScript. Pour vraiment couvrir les 5 cursus, il manque : des missions de data science avec datasets réels (pas juste du code Python générique), des missions d'IA qui incluent l'entraînement de modèles simples, et des missions de dev web qui incluent du HTML/CSS en plus du JavaScript. L'extension peut se faire progressivement, un langage à la fois, en commençant par celui qui vous sert le plus immédiatement.

Mnemo est un projet personnel d'apprentissage. Le code source est disponible, itérable, et conçu pour évoluer avec son créateur. La présente architecture est une base de discussion, pas un contrat — chaque décision technique doit être remise en question face à l'usage réel.